UD 6

Interfaces

–

Módulo de Programación
1º DAW

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Autor: Fran Gómez
2025/2026

Objetivos

Comprender el uso de interfaces en Java como mecanismo fundamental para definir contratos, favorecer el **polimorfismo** y mejorar el diseño **modular** de aplicaciones.

- **Entender** el concepto de interfaz como contrato dentro de la programación orientada a objetos.
- **Diferenciar** correctamente entre interfaces y clases abstractas.
- **Definir** interfaces utilizando la sintaxis propia de Java.
- **Implementar** interfaces en clases concretas.
- **Aplicar** el polimorfismo mediante el uso de interfaces.

Objetivos

Comprender el uso de interfaces en Java como mecanismo fundamental para definir contratos, favorecer el **polimorfismo** y mejorar el diseño **modular** de aplicaciones.

- **Diseñar** jerarquías de interfaces y comprender la herencia múltiple de tipo.
- **Utilizar** métodos default y static en interfaces.
- **Aplicar** buenas prácticas de diseño orientado a interfaces.
- **Identificar** y evitar errores comunes en el uso de interfaces.
- **Resolver** problemas prácticos mediante el uso de interfaces en Java.

Índice de contenidos resumido

1. Introducción a las Interfaces
2. Definición de una Interfaz en Java
3. Implementación de Interfaces
4. Interfaces vs Clases Abstractas
5. Polimorfismo con Interfaces
6. Interfaces y Herencia
7. Métodos en Interfaces (Java moderno)
8. Buenas prácticas
9. Ejemplo completo guiado
10. Errores comunes
11. Actividad práctica
12. Resumen y conclusiones

¿Qué se va a evaluar?

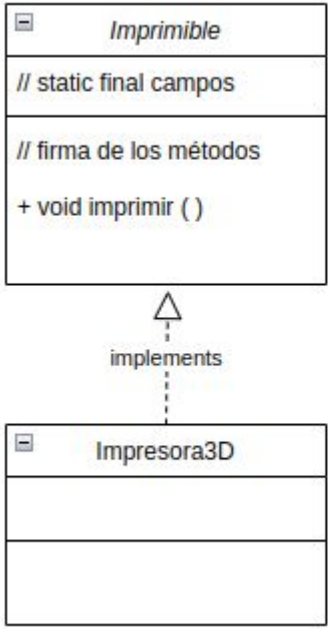
	Mod.	RA
RA 4 Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.	10%	
i) Se han definido y utilizado interfaces.	1%	10%

¿Cómo lo vamos a evaluar?

Instrumento	Técnica
Portfolio: - Se evalúa el trabajo diario - Recurso para estudiar - Útil como CV	- Salir a corregir los ejercicios de clase - Defender los ejercicios necesarios frente al profesor antes del examen.
Proyecto: - Evaluación de todo lo aprendido y más - Aplicación en diferentes contextos - Colaborativo	- Exposición: explicar qué y cómo funciona a la clase, respondiendo a las preguntas.
Examen: - Evaluación individual y sumativa de lo fundamental	- Prueba escrita

Interfaces

Plantilla que define un conjunto de métodos abstractos y constantes → Sin implementaciones




```
public interface Imprimible {
    public void imprimir ( );
}
```

¿Qué?

```
public class Impresora3D implements Imprimible {
    public void imprimir ( ) {
        // Código
    }
}
```

¿Cómo?

- Otra forma de lograr **Abstracción**
- **Métodos abstractos**: Declarados sin cuerpo, deben ser implementados por las clases. (No se pone abstract)
- **Constantes**: Son implícitamente `public`, `static` y `final`.
- Puede ser **implementada** por clases
- **Herencia múltiple**: Una clase puede implementar varias interfaces.
- Desde Java 8: Las interfaces pueden tener métodos por defecto y métodos estáticos con implementación.
- No puede ser instanciada
- Sí puede ser extendida por otras interfaces

Interfaces vs Clases abstractas

	Interfaz	Clase abstracta
Métodos	**Solo abstractos** (hasta Java 7). Desde Java 8, también por defecto y estáticos.	Abstractos y concretos.
Herencia	Herencia múltiple	Herencia simple
Modificadores de acceso	public o default(package)	Puede tener cualquier modificador
Usos	Clases **no** están muy **relacionadas** pero queremos añadirles funcionalidades comunes. Interfaces abiertas a menos cambios.	Clases están **relacionadas** entre sí. Clases abierta a más cambios.

Interfaces: sintaxis

public **interface** <nombre> [extends Interface1, Interface2…] {

// Declaración de **constantes** → son **public static final** por defecto

// Declaración de **métodos** sin cuerpo → son **public abstract** por defecto

}

- Misma convención de nombres que para las clases
- public o package
- Si es public, el nombre del fichero debe llamarse igual que la interface

Interfaces: Ejemplo

```java
public interface Interface {
    void metodo ();
}
```

Implements

Implements

```java
public class ImplementacionA implements Interface {

    @Override
    public void metodo() {
        System.out.println("ImplementacionA");
    }
}
```

```java
public class ImplementacionB implements Interface {

    @Override
    public void metodo() {
        System.out.println("ImplementacionB");
    }
}
```

Ejemplo completo: la interfaz


```java
// Definición de la interfaz
public interface Vehiculo {

    // Atributo (constante)
    int VELOCIDAD_MAXIMA = 120;

    // Métodos abstractos
    void acelerar(int cantidad);
    void frenar(int cantidad);

    // Método default (Java 8+)
    default void mostrarVelocidadMaxima() {
        System.out.println("Velocidad máxima: " + VELOCIDAD_MAXIMA);
    }

    // Método estático
    static void informacionGeneral() {
        System.out.println("Los vehículos permiten desplazarse.");
    }
}
```

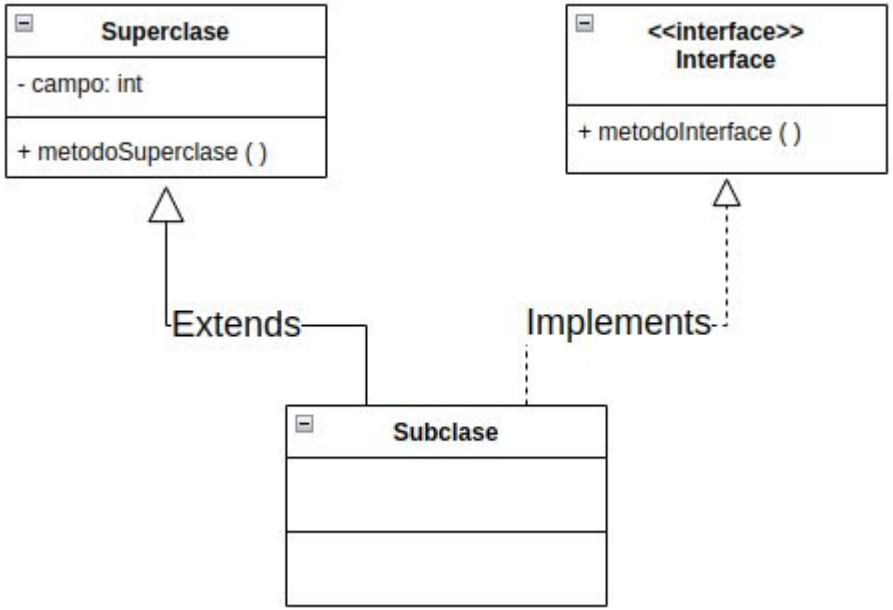
Ejemplo completo: la implementación

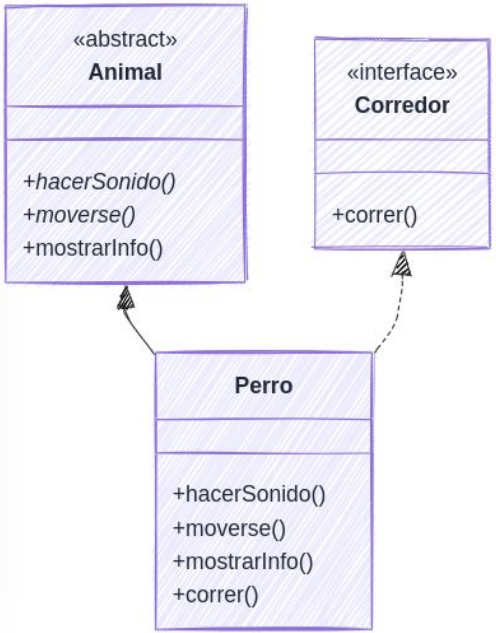

```java
public class Coche implements Vehiculo {

    private int velocidadActual;

    @Override
    public void acelerar(int cantidad) {
        velocidadActual += cantidad;
        if (velocidadActual > VELOCIDAD_MAXIMA) {
            velocidadActual = VELOCIDAD_MAXIMA;
        }
    }

    @Override
    public void frenar(int cantidad) {
        velocidadActual -= cantidad;
        if (velocidadActual < 0) {
            velocidadActual = 0;
        }
    }

    public void mostrarEstado() {
        System.out.println("Velocidad actual: " + velocidadActual);
    }
}
```

Ejemplo completo: el uso


```java
public class Main {
    public static void main(String[] args) {

        Coche coche = new Coche();

        coche.acelerar(50);
        coche.mostrarEstado();

        coche.mostrarVelocidadMaxima(); // método default

        Vehiculo.informacionGeneral(); // método static
    }
}
```

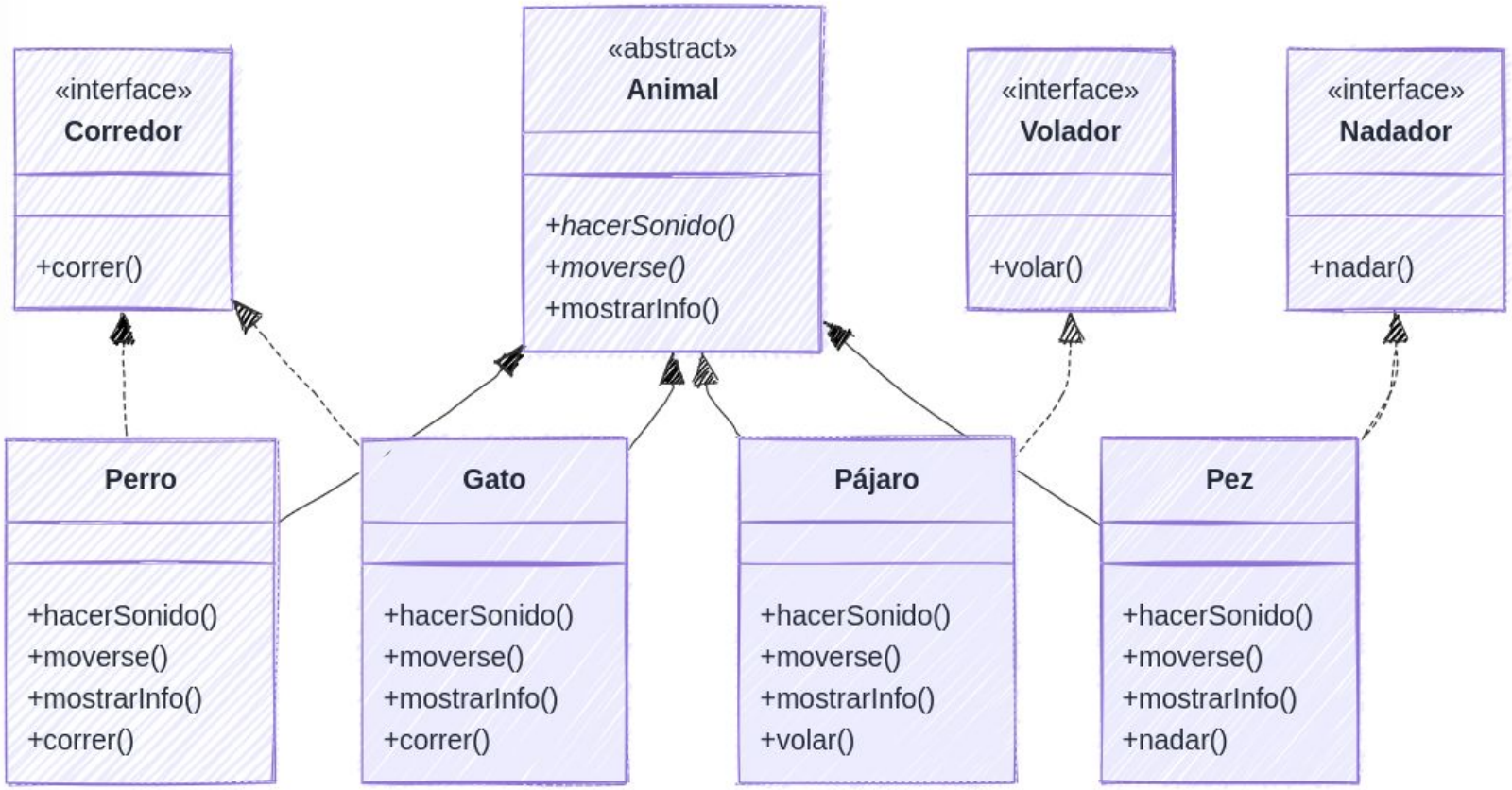
Herencia e interfaces juntas






```
class Perro extends Animal implements Corredor {
    …
}
```

Ejercicio 1


«interface»
Corredor
+correr()
«abstract»
Animal
+hacerSonido()
+moverse()
+mostrarInfo()
«interface»
Volador
+volar()
«interface»
Nadador
+nadar()
Perro
+hacerSonido()
+moverse()
+mostrarInfo()
+correr()
Gato
+hacerSonido()
+moverse()
+mostrarInfo()
+correr()
Pájaro
+hacerSonido()
+moverse()
+mostrarInfo()
+volar()
Pez
+hacerSonido()
+moverse()
+mostrarInfo()
+nadar()

Buenas prácticas

★ Se declara con Interfaces y se Instancia con Clases

```
Animal mascota = new Perro();
```

Así se favorece un código más flexible gracias al **polimorfismo**

Interfaces y Polimorfismo


```java
interface Animal {
    void hacerSonido();
}

class Perro implements Animal {
    public void hacerSonido() {
        System.out.println("Guau");
    }
}

class Gato implements Animal {
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

```java
public class Main {
public static void main(String[] args)
{
    Animal a;

    a = new Perro();
    a.hacerSonido(); // Guau

    a = new Gato();
    a.hacerSonido(); // Miau
}
}
```

Ejercicio 2

Actividad: Sistema de animales (polimorfismo + interfaces + colecciones)

Contexto

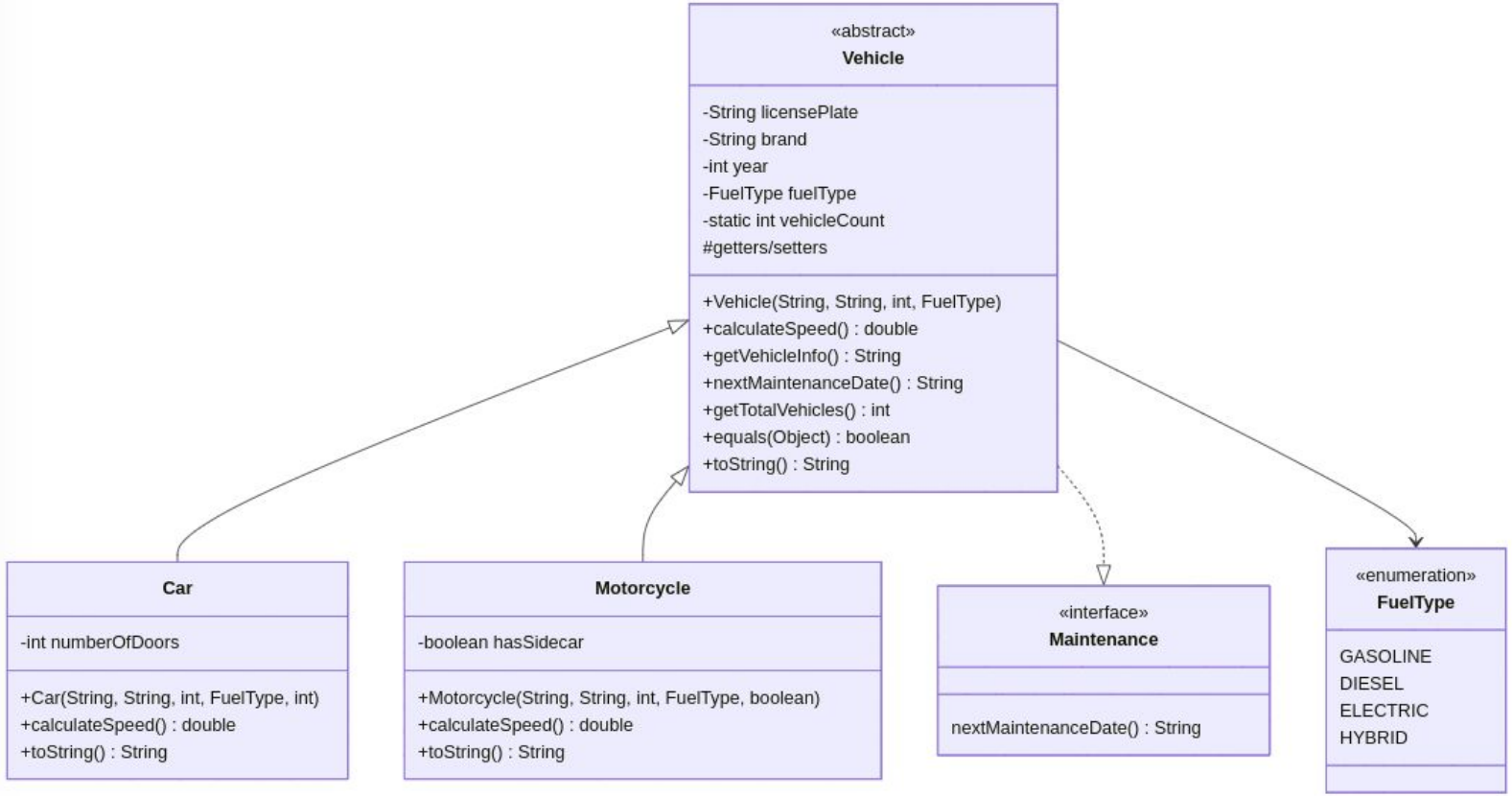
Se quiere desarrollar un pequeño sistema que gestione distintos tipos de animales en una granja/zoológico. Todos los animales deben poder emitir un sonido, pero cada uno lo hará de forma diferente.

Your Wooclap question will appear here

1. Install the **Chrome** or **Firefox** extension 🧩
2. Click on “Add a Wooclap vote” and make sure you are logged into your Wooclap account 🧑‍💻
3. Ensure you are in **presentation mode** on Google Slides 🖥️

wooclap

Ejercicio tipo examen. Ejercicio 3



Recursos y referencias

- Libro que tenéis en la biblioteca y en el departamento
- Diapositivas en Moodle
- Glosario en Moodle
- Ejercicios de refuerzo y de ampliación
- Vuestro portfolio